

Tarea 1 - Sistemas Operativos

Implementación de una Shell Simple con Pipes, Redirección y Manejo de Señales

Benjamín Díaz Villalobos

28 de Septiembre de 2026

1 Introduction

En el siguiente informe se presenta la implementación de una shell simple de texto en C (POSIX) para sistemas Linux. El objetivo central es construir un intérprete de comandos funcional que actúe como interfaz entre el usuario y el núcleo, gestionando el ciclo de vida de los procesos y sus recursos mediante llamadas al sistema para el control de procesos, la manipulación y redirección de descriptores de archivo y la gestión asíncrona de señales. Asimismo, se detalla la implementación de comandos internos y la construcción de la herramienta de monitoreo "pmon", basada en la lectura e implementación del pseudo-sistema de archivos.

Repositorio de código: <https://github.com/Benjinho10/Sistemas-Operativos—Tarea-1>

2 Arquitectura General de la Shell

La shell funciona mediante el ciclo interactivo REPL (*Read-Eval-Print Loop*). Antes de mostrar el prompt en pantalla, la shell realiza una revisión de la lista interna de trabajos para ver si han terminado en segundo plano. Si terminó, imprime el aviso correspondiente. Luego obtiene la ruta actual con `getcwd()` y muestra el prompt `<directorio>$`. Al ser ingresada una línea de comandos por el usuario, `fgets()` lee la línea y la descompone en tokens usando `strtok()`, separando por espacios y tabulaciones. De esta forma también se identifican los operadores especiales, tales como `&` (background), `|` (pipes) o si se usan `<`, `>`, `>>` (redireccionadores de archivos).

Luego la shell entra en un proceso de clasificación en función de la estructura de la entrada: si es un comando interno (tales como `cd`, `exit`, `jobs` o `pmon`), la shell lo ejecuta directamente en su propio proceso; si contiene pipes, delega la ejecución a la función encargada de encadenar los comandos; y si es un comando simple externo, crea un proceso hijo con `fork()`, aplica las redirecciones de archivo si es necesario y carga el programa con `execvp()`, mientras que el proceso padre invoca `waitpid()` para esperar su finalización y completar el ciclo de sincronización adecuadamente.

Para rastrear los procesos en el background se definió un arreglo global de estructuras llamado `jobs[]` con una capacidad acotada definida por `MAX_JOBS`, en donde cada entrada almacena: `id`, correspondiente al número secuencial visible para el usuario; `pid`, el identificador real asignado por el sistema operativo al proceso hijo; `cmd`, una copia íntegra del comando original ejecutado; y `terminado`, una bandera que indica con 0 si continúa en ejecución o 1 si el manejador de señales ya detectó su término.

3 Implementación de Tuberías de Largo Arbitrario

Para encadenar cualquier cantidad de comandos (`cmd1 | cmd2 | ... | cmdN`), el programa utiliza un bucle iterativo apoyado en una variable clave denominada `prev_fd` (descriptor de lectura del paso anterior) para evitar fijar un número estático de procesos. En cada vuelta del ciclo, si el comando actual no es el último de la fila, se crea una tubería nueva llamando a `pipe(pipefd)`, lo que genera dos extremos: `pipefd[0]` para operaciones de lectura y `pipefd[1]` para escritura.

En el proceso hijo creado con `fork()`, la configuración se bifurca según su posición: si no es el primer comando (es decir, `prev_fd` existe), redirige su entrada estándar para leer desde `prev_fd` utilizando `dup2(prev_fd, STDIN_FILENO)` y cierra `prev_fd`. A su vez, si no es el último comando, redirige su salida estándar hacia el extremo de escritura de la tubería recién creada usando `dup2(pipefd[1], STDOUT_FILENO)`, cerrando ambos extremos locales. Cuando

los canales quedan debidamente conectados a las tuberías, el hijo ejecuta el comando correspondiente llamando a `execvp()`.

En el proceso padre, si existía un `prev_fd` abierto de la iteración anterior, procede a cerrarlo con `close()` puesto que dicho comando previo ya no procesará más datos. Si no era el último comando, cierra de inmediato el extremo de escritura `pipefd[1]` para asegurar el envío de la señal de fin de archivo (EOF) al lector al culminar la emisión, y almacena el extremo de lectura `pipefd[0]` en `prev_fd` para conectárselo al siguiente hijo en la próxima vuelta. Finalmente, si la tubería se lanzó en primer plano, la shell ejecuta un bucle con `waitpid()` esperando a que todos los PIDs de la cadena terminen antes de devolver el control al usuario; mientras que si se indicó ejecución en background (`&`), simplemente registra el último PID en la lista de jobs y continúa con el flujo habitual.

4 Manejo de Señales

Para la gestión de señales se utilizó la llamada al sistema `sigaction()`, garantizando control estricto sobre las máscaras de bloqueo y los atributos de ejecución. Al iniciar la ejecución, la shell configura las señales `SIGINT` (Ctrl+C) y `SIGQUIT` (Ctrl+\) bajo la disposición `SIG_IGN` con el flag `SA_RESTART`. Esto asegura que presionar dichas combinaciones en un prompt vacío no termine el intérprete ni interrumpa de forma abrupta las operaciones de lectura en la entrada estándar.

Cuando la shell invoca `fork()` para instanciar un proceso hijo, este hereda inicialmente la disposición de ignorar señales. Por esta razón, antes de ejecutar la llamada `execvp()`, se evalúa el contexto de ejecución: si el comando corre en primer plano, se restablece `SIGINT` a su comportamiento por defecto (`SIG_DFL`), permitiendo al usuario interrumpir el comando activo con Ctrl+C sin perjudicar a la shell padre; en contraste, si el proceso corre en segundo plano (`&`), se preserva la disposición `SIG_IGN` para evitar que las señales emitidas desde la terminal afecten tareas en background.

Por último, cada vez que un proceso hijo concluye, el kernel notifica a la shell mediante la señal `SIGCHLD`. Dado que en Linux las señales estándar no se encolan, el manejador implementa un ciclo iterativo mediante `while ((pid = waitpid(-1, &status, WNOHANG)) > 0)`. La bandera `WNOHANG` resulta fundamental al evitar que la llamada bloquee a la shell cuando no quedan más procesos listos, permitiendo recolectar a todos los hijos finalizados en una única ráfaga y previniendo la acumulación de procesos en estado zombie.

5 Diseño del Monitor de Procesos (pmon)

El comando interno `pmon` opera como un monitor en tiempo real de los procesos en segundo plano leyendo directamente desde el sistema de archivos virtual `/proc`, sin depender de herramientas externas del sistema. La extracción de datos se realiza abriendo `/proc/[pid]/stat` para consultar el estado y el uso de CPU. Dado que el nombre del ejecutable puede contener espacios o caracteres especiales encerrados en paréntesis, el analizador localiza el último paréntesis de cierre (')') para leer a partir de allí el estado del proceso (`state`) y los tiempos acumulados en modo usuario (`utime`) y modo kernel (`stime`). Paralelamente, se examina `/proc/[pid]/status` recorriendo sus líneas hasta identificar la etiqueta `VmRSS:`, obteniendo la memoria física residente ocupada en kilobytes.

El cálculo del porcentaje de CPU se implementa mediante un esquema diferencial basado en los ticks de reloj acumulados. En cada iteración se determina la variación temporal restando la sumatoria previa de `utime + stime` a la lectura actual. Dicho incremento de ticks se transforma a segundos al dividirse por la frecuencia del reloj del sistema, obtenida en tiempo de ejecución mediante la llamada `sysconf(_SC_CLK_TCK)`. Finalmente, el tiempo efectivo de CPU consumido se divide por el intervalo de refresco configurado y se multiplica por 100 para computar la tasa de uso proporcional.

El refresco periódico se controla con `alarm(intervalo)`, la cual emite la señal `SIGALRM` al expirar el temporizador. El manejador de esta señal conmuta una bandera segura (`volatile sig_atomic_t pmon_tick = 1`), desbloqueando la llamada `pause()` para redibujar la tabla de métricas en pantalla. Adicionalmente, se instala un manejador temporal para `SIGINT` que intercepta Ctrl+C y altera una bandera de terminación, permitiendo salir limpiamente del bucle de monitoreo, restaurar los manejadores originales y devolver el control al prompt interactivo sin clausurar la shell ni perturbar los procesos en background.

6 Pruebas Realizadas

Se llevaron a cabo pruebas empíricas sobre cada funcionalidad descrita en la guía para validar la robustez del sistema:

- Se ejecutó `ls -l | grep ".c" | wc -l`, verificando la correcta propagación de descriptores a través de `pipe()` y `dup2()`, y comprobando que la shell padre espera la finalización completa de la secuencia antes de retornar el prompt.
- Se validaron los operadores mediante `echo "test" > f.txt`, `echo "linea" >> f.txt` y `wc -l < f.txt`, comprobando los modos de truncado, anexo y lectura sin afectar los descriptores estándar del intérprete.
- Se lanzaron tareas asíncronas con `sleep 20 &`, listándolas con el comando interno `jobs`. Tras expirar el tiempo, la shell notificó oportunamente su término en el siguiente ciclo REPL. Mediante inspección externa con `ps aux`, se constató la ausencia total de procesos en estado zombie (*defunct*) gracias a la recolección con `WNOHANG`.
- Se comprobó la inmunidad de la shell al pulsar Ctrl+C (SIGINT) en un prompt vacío. Al ejecutar `sleep 60` en primer plano, Ctrl+C interrumpió de inmediato al proceso hijo sin finalizar la shell padre ni afectar a tareas que corrían en segundo plano.
- Se ejecutó `pmon 2` tras iniciar cargas intensivas en background (`yes > /dev/null &`), verificando la correcta lectura de memoria física desde `/proc/[pid]/status` y el cálculo diferencial dinámico de CPU desde `/proc/[pid]/stat`. Se validó además la salida limpia al prompt mediante Ctrl+C.

7 Limitaciones de la Implementación

Entre las principales limitaciones identificadas en la versión actual de la shell se encuentran las siguientes:

Las redirecciones de archivo (`<`, `>`, `>>`) únicamente están procesadas en el bloque correspondiente a comandos simples dentro de `main()`, por lo que una instrucción que combine redirecciones y tuberías simultáneamente no aplica los descriptores de archivo en los extremos del pipeline.

Existe una capacidad fija para los argumentos y la tabla de procesos debido al uso de memoria estática (longitud máxima de línea acotada a 1024 bytes, máximo de 64 argumentos y un tope de 64 trabajos en segundo plano definidos por constantes), lo que implica que entradas con argumentos masivos o una sobrecarga de procesos en background provocarán la saturación de los buffers internos.

El análisis léxico emplea la función `strtok(line, " \t")`, lo que impide la correcta interpretación y agrupación de cadenas compuestas con espacios mediante comillas dobles o simples, fragmentándolas de forma errónea en múltiples tokens independientes.

No se implementó el control de grupos de procesos mediante llamadas como `tcsetpgrp()` para la suspensión de comandos con Ctrl+Z (SIGTSTP), omitiendo la incorporación de los comandos internos `fg` o `bg` para la alternancia de tareas entre primer y segundo plano.

8 Preguntas de reflexión

1. **¿Por qué `cd` y `exit` deben implementarse como comandos internos y no mediante `fork()` y `exec()`?**

Los comandos `cd` y `exit` deben implementarse como comandos internos porque modifican directamente el estado interno y el ciclo de vida del proceso de la shell. Si `cd` se ejecutara en un proceso hijo, la llamada al sistema `chdir()` cambiaría únicamente el directorio de trabajo de ese hijo dentro de su propio espacio de memoria aislado. Al terminar el hijo, la shell padre continuaría en la misma carpeta donde estaba. Lo mismo sucede con `exit`, su objetivo es cerrar la shell por completo, pero si se invocara mediante un proceso hijo, sería dicho proceso quien terminaría su ejecución, dejando a la shell viva sin que el programa se cierre.

2. **En una tubería `cmd1 | cmd2 | cmd3`, ¿qué pasaría si no se cerraran los descriptores de pipe no usados en cada proceso? Describa un escenario concreto de falla.**

Si la shell padre o alguno de los procesos deja abierto un descriptor de escritura que no está usando, el contador de referencias del kernel nunca llega a cero y el proceso lector jamás recibe el fin de archivo (*EOF*), el cual indica que todos los extremos de escritura de esa tubería han sido cerrados en el sistema.

Consideremos el comando `cat archivo.txt | grep "error" | wc -l`. Si la shell padre olvida cerrar en su propio código el extremo de escritura de la segunda tubería (que conecta `grep` con `wc`), el sistema operativo detectará que todavía queda un escritor vivo. En consecuencia, `wc -l` procesará los datos que alcanzó a enviar `grep`, pero al llamar a `read()` para continuar, quedará bloqueado indefinidamente esperando datos que nunca llegarán. La tubería jamás finaliza y la terminal se queda congelada.

3. **¿Por qué el manejador de SIGCHLD debe llamar a waitpid() en un ciclo con WNOHANG, en vez de una sola vez?**

El manejador de SIGCHLD debe ejecutar `waitpid()` dentro de un ciclo `WNOHANG` debido a que en Linux las señales estándar no se encolan. Si múltiples procesos hijos terminan de forma casi simultánea, el sistema operativo fusiona esas notificaciones y le entrega a la shell una única señal SIGCHLD. Si se llamara a `waitpid()` una sola vez, se recolectaría solo el primer proceso que finalizó, dejando a los demás atrapados indefinidamente en la tabla de procesos del sistema en estado zombie. Asimismo, se requiere la flag `WNOHANG` para que la llamada no sea bloqueante, evitando que la shell se quede congelada si aún quedan otros procesos ejecutándose en segundo plano.

4. **¿Qué diferencia de comportamiento observarían si, en vez de restaurar SIGINT a su disposición por defecto en el hijo, simplemente no tocan la disposición heredada de la shell?**

Cuando clonamos un proceso, el proceso hijo hereda la tabla de manejadores del padre. Cuando luego se sobrescribe la memoria del programa en el proceso hijo, los manejadores vuelven a su valor por defecto, pero las señales configuradas como ignoradas preservan su estado a través de `execvp()`.

Si la shell padre ignora SIGINT y no restaura su acción por defecto (en este caso SIG_DFL) en el hijo antes de invocar `execvp()`, el comando ejecutado en primer plano heredará SIG_IGN. Esto se vería representado en la terminal, ya que no podría interrumpirse o cancelarse algún comando previamente ejecutado, perdiendo el control sobre procesos estancados o de larga duración.

5. **En pmon, ¿qué ocurre si un proceso monitoreado termina justo entre dos lecturas de /proc? ¿Cómo lo manejó su implementación?**

Si un proceso en segundo plano finaliza su ejecución entre dos lecturas, el kernel elimina las entradas asociadas a dicho PID una vez que es recolectado. Si el código intentara leer esas rutas asumiendo que siempre existen, llamadas al sistema como `fopen()` fallarían retornando un puntero NULL.

Para manejar esta situación, en la implementación se cuenta con la validación de apertura de archivos, en donde funciones como `obtener_stat_proc()` y `obtener_rss_proc()` verifican siempre el valor retornado por `fopen()`. Si el archivo no se puede abrir, la función finaliza de inmediato sin intentar leer la memoria ni parsear buffers vacíos.

También se presenta el filtrado dinámico en la visualización, en donde en cada ciclo de actualización de la función `ejecutar_pmon()` se asume que el proceso no está disponible en el sistema cuando `obtener_stat_proc()` retorna 0, omitiendo el cálculo del %CPU y evitando imprimir datos inconsistentes en la tabla.

Finalmente, el manejador de SIGCHLD recoge el proceso con `waitpid(-1, &status, WNOHANG)` y actualiza la bandera en la estructura de jobs, garantizando que en el siguiente ciclo el proceso quede completamente fuera de la lista de monitoreo.